CH VASU
2303A51618
B25

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTM	
Program Name: B. Tech		**Assignment Type: Lab**	
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
CourseCode	23CS002PC304	**Course Title**	AI
Year/Sem	III/II	**Regulation**	R
Date and Day of Assignment	**Week6 – Wednusday**	**Time(s)**	23
Duration	2 Hours	**Applicable to Batches**	A
Assignment Number:12.3(Present assignment number)/**24**(Total number of assignments)			

Question

Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tool

Lab Objectives

The objectives of this laboratory exercise are to:

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive

1 **Scenario**

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placemen
descending order.

Tasks

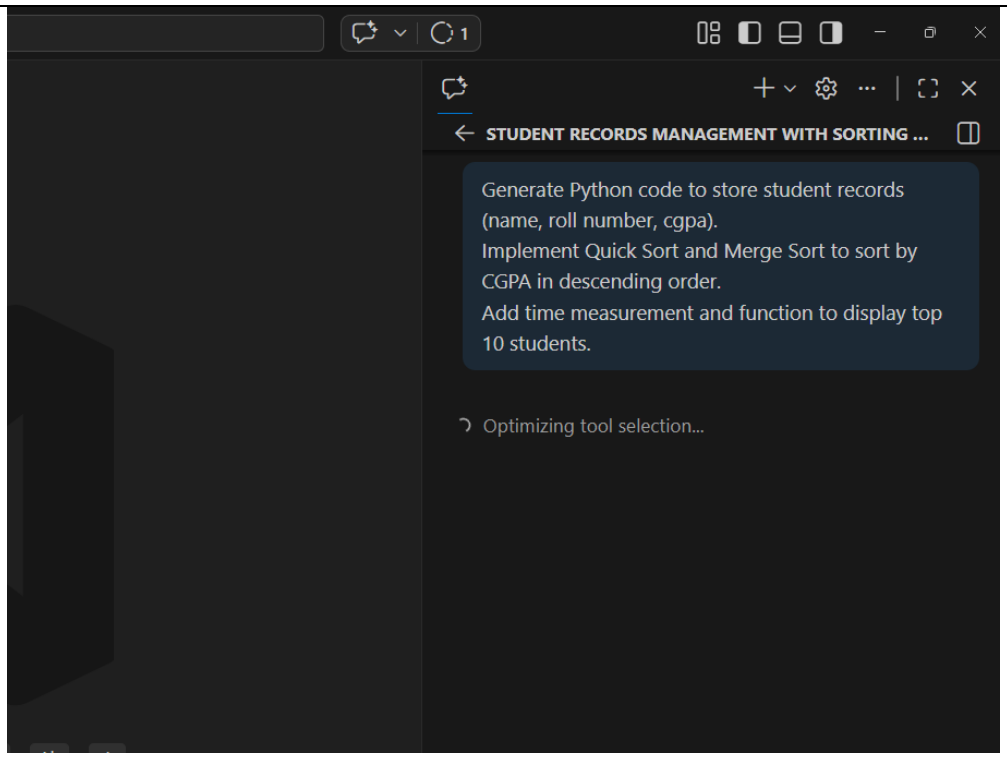
1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
   - Quick Sort
   - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

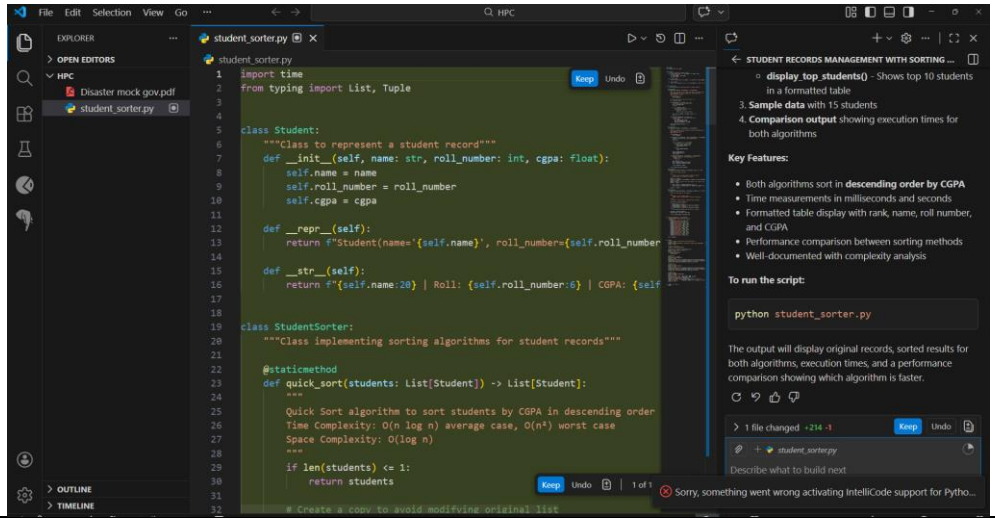
Expected Outcome

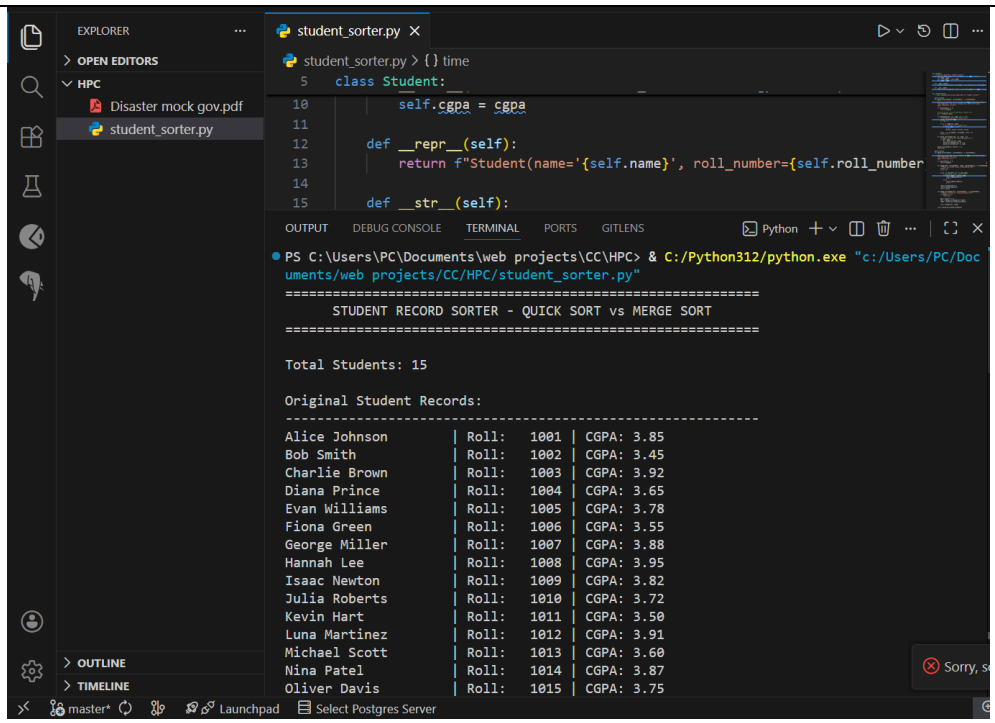
- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

Prompt

Generate Python code to store student records (name, roll number, cgpa).
Implement Quick Sort and Merge Sort to sort by CGPA in descending order.
Add time measurement and function to display top 10 students.


STUDENT RECORDS MANAGEMENT WITH SORTING ...
Generate Python code to store student records (name, roll number, cgpa).
Implement Quick Sort and Merge Sort to sort by CGPA in descending order.
Add time measurement and function to display top 10 students.
Optimizing tool selection...


File Edit Selection View Go
EXPLORER
OPEN EDITORS
HPC
Disaster mock gov.pdf
student_sorter.py
OUTLINE
TIMELINE
student_sorter.py
import time
from typing import List, Tuple
class Student:
"""Class to represent a student record"""
def __init__(self, name: str, roll_number: int, cgpa: float):
self.name = name
self.roll_number = roll_number
self.cgpa = cgpa
def __repr__(self):
return f"Student(name='{self.name}', roll_number={self.roll_number
def __str__(self):
return f"{self.name:20} | Roll: {self.roll_number:6} | CGPA: {self
class StudentSorter:
"""Class implementing sorting algorithms for student records"""
@staticmethod
def quick_sort(students: List[Student]) -> List[Student]:
"""
Quick Sort algorithm to sort students by CGPA in descending order
Time Complexity: O(n log n) average case, O(n²) worst case
Space Complexity: O(log n)
"""
if len(students) <= 1:
return students
Keep
Undo
STUDENT RECORDS MANAGEMENT WITH SORTING ...
display_top_students() - Shows top 10 students in a formatted table
3. Sample data with 15 students
4. Comparison output showing execution times for both algorithms
Key Features:
Both algorithms sort in descending order by CGPA
Time measurements in milliseconds and seconds
Formatted table display with rank, name, roll number, and CGPA
Performance comparison between sorting methods
Well-documented with complexity analysis
To run the script:
python student_sorter.py
The output will display original records, sorted results for both algorithms, execution times, and a performance comparison showing which algorithm is faster.
1 file changed +214 -1
Describe what to build next
Sorry, something went wrong activating IntelliCode support for Pytho...


EXPLORER
OPEN EDITORS
HPC
Disaster mock gov.pdf
student_sorter.py
student_sorter.py
student_sorter.py > { } time
class Student:
self.cgpa = cgpa
def __repr__(self):
return f"Student(name='{self.name}', roll_number={self.roll_number
def __str__(self):
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
GITLENS
Python
PS C:\Users\PC\Documents\web projects\CC\HPC> & C:/Python312/python.exe "c:/Users/PC/Doc
uments/web projects/CC/HPC/student_sorter.py"
STUDENT RECORD SORTER - QUICK SORT vs MERGE SORT
Total Students: 15
Original Student Records:
Alice Johnson | Roll: 1001 | CGPA: 3.85
Bob Smith | Roll: 1002 | CGPA: 3.45
Charlie Brown | Roll: 1003 | CGPA: 3.92
Diana Prince | Roll: 1004 | CGPA: 3.65
Evan Williams | Roll: 1005 | CGPA: 3.78
Fiona Green | Roll: 1006 | CGPA: 3.55
George Miller | Roll: 1007 | CGPA: 3.88
Hannah Lee | Roll: 1008 | CGPA: 3.95
Isaac Newton | Roll: 1009 | CGPA: 3.82
Julia Roberts | Roll: 1010 | CGPA: 3.72
Kevin Hart | Roll: 1011 | CGPA: 3.50
Luna Martinez | Roll: 1012 | CGPA: 3.91
Michael Scott | Roll: 1013 | CGPA: 3.60
Nina Patel | Roll: 1014 | CGPA: 3.87
Oliver Davis | Roll: 1015 | CGPA: 3.75
OUTLINE
TIMELINE
master*
Launchpad
Select Postgres Server

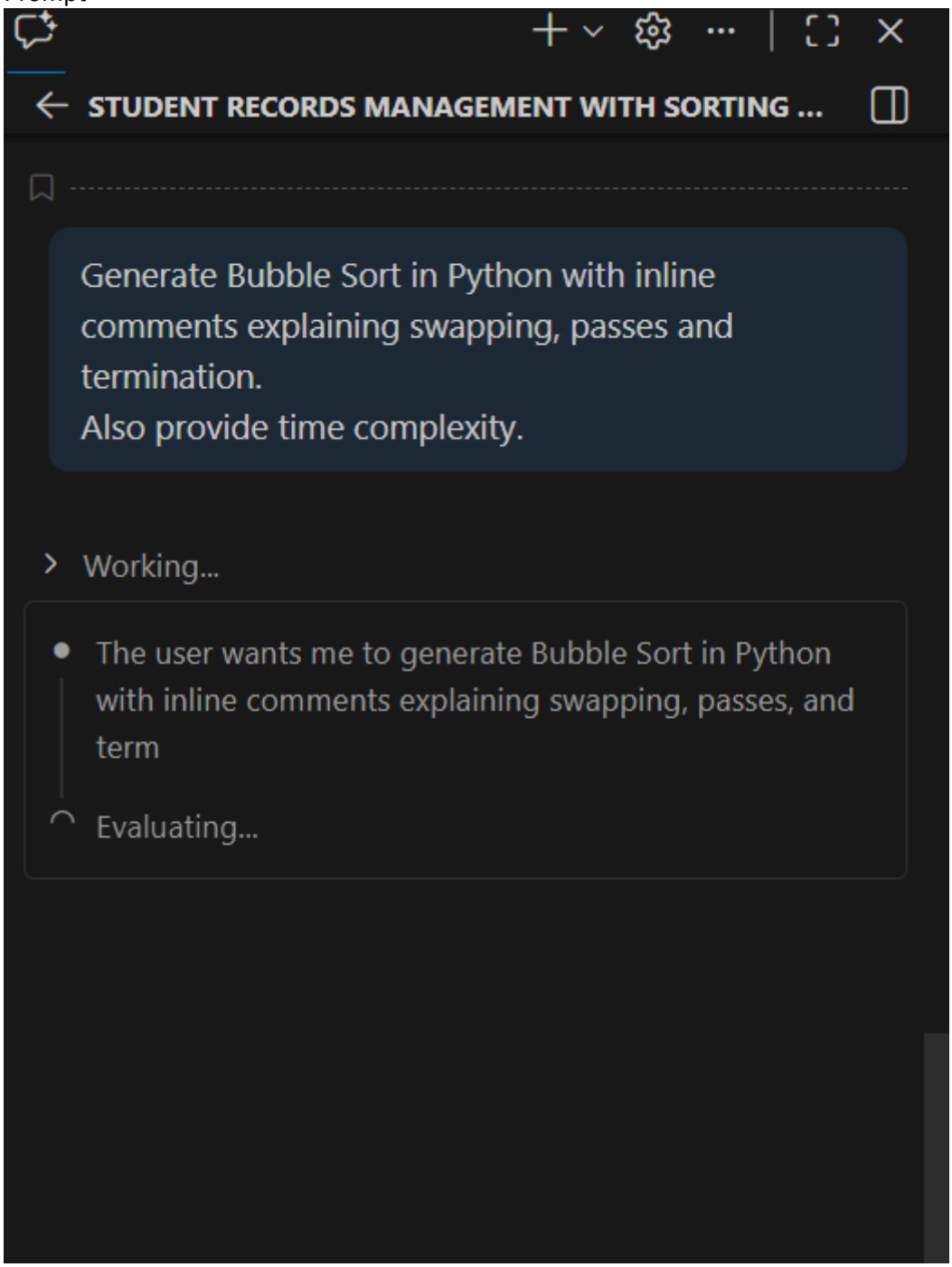
```
PS C:\Users\PC\Documents\web projects\CC\HPC> & C:/Python312/python.e
uments/web projects/CC/HPC/student_sorter.py"
Rank    Name                 Roll Number    CGPA
------------------------------------------------------------
1       Hannah Lee           1008           3.95
2       Charlie Brown        1003           3.92
3       Luna Martinez        1012           3.91
4       George Miller        1007           3.88
5       Nina Patel           1014           3.87
6       Alice Johnson        1001           3.85
7       Isaac Newton         1009           3.82
8       Evan Williams        1005           3.78
9       Oliver Davis         1015           3.75
10      Julia Roberts        1010           3.72
============================================================

============================================================
PERFORMANCE COMPARISON
============================================================
Quick Sort Time:  0.0778 ms
Merge Sort Time:  0.1091 ms
Difference:       0.0313 ms (Quick Sort is faster)
============================================================
PS C:\Users\PC\Documents\web projects\CC\HPC>
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.
- **Instructions**:
  - Students implement Bubble Sort normally.
  - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termin
  - Request AI to provide **time complexity analysis**.
- **Expected Output**:
  - A Bubble Sort implementation with AI-generated explanatory comments and complexity ana

Prompt


STUDENT RECORDS MANAGEMENT WITH SORTING ...
Generate Bubble Sort in Python with inline comments explaining swapping, passes and termination.
Also provide time complexity.
Working...
The user wants me to generate Bubble Sort in Python with inline comments explaining swapping, passes, and term
Evaluating...

File Edit Selection View Go ··· HPC

EXPLORER

OPEN EDITORS

HPC

Disaster mock gov.pdf

student_sorter.py

student_sorter.py > { } time

```python
class StudentSorter:
    @staticmethod
    def bubble_sort(students: List[Student]) -> List[Student]:
        """
        Bubble Sort algorithm to sort students by CGPA in descending order

        Algorithm: Repeatedly steps through the list, compares adjacent el
        them if they are in the wrong order. Process repeats until no more

        Time Complexity: O(n²) - Best case O(n), Average case O(n²), Worst
        Space Complexity: O(1) - Sorts in-place
        """
        # Create a copy to avoid modifying original list
        arr = students.copy()
        n = len(arr)

        # Outer loop: number of passes through the array
        # Each pass moves one element to its correct
        for i in range(n):
            # Flag to optimize: if no swaps occur, array is already sorted
```

Keep Undo 1 of 7

OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PowerShell Extension v2025.4.0
Copyright (c) Microsoft Corporation.

https://aka.ms/vscode-powershell
Type 'help' to get help.

PS C:\Users\PC\Documents\web projects\CC\HPC>
```

Python

PowerShell Extension

OUTLINE

TIMELINE

```
PS C:\Users\PC\Documents\web projects\CC\HPC> cd "c:\Users
ts\CC\HPC" ; python student_sorter.py
Rank   Name                 Roll Number    CGPA
------------------------------------------------------------
1      Hannah Lee           1008           3.95
2      Charlie Brown        1003           3.92
3      Luna Martinez        1012           3.91
4      George Miller        1007           3.88
5      Nina Patel           1014           3.87
6      Alice Johnson        1001           3.85
7      Isaac Newton         1009           3.82
8      Evan Williams        1005           3.78
9      Oliver Davis         1015           3.75
10     Julia Roberts        1010           3.72
============================================================


============================================================
PERFORMANCE COMPARISON
============================================================
Quick Sort Time:   0.0852 ms   - O(n log n) average
Merge Sort Time:   0.0980 ms   - O(n log n) guaranteed
Bubble Sort Time:  0.0518 ms   - O(n²)
------------------------------------------------------------
Fastest: Bubble Sort (0.0518 ms)
============================================================
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion.
- **Instructions:**
  - Provide AI with partially completed functions for recursion.
  - Ask AI to complete the missing logic and add docstrings.
  - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
  - Working Quick Sort and Merge Sort implementations.
  - AI-generated explanation of average, best, and worst-case complexities.

Prompt

Complete recursive quick sort and merge sort functions in Python.
Add docstrings and complexity explanation.
Compare results on random, sorted and reverse sorted lists.

STUDENT RECORDS MANAGEMENT WITH SORTING ...

Complete recursive quick sort and merge sort functions in Python.
Add docstrings and complexity explanation.
Compare results on random, sorted and reverse sorted lists.

Working...

Let me create a comprehensive Python file that demonstrates both algorithms with:

- Detailed docstrings
- Time and space complexity explanations
- Test cases with random, sorted, and reverse sorted data
- Performance comparisons across different data patterns

Creating recursive_sorting.py (425 lines)

student_sorter.py

Describe what to build next

Auto

```
PS C:\Users\PC\Documents\web projects\CC\HPC> cd "c:\Users\PC\
ts\CC\HPC" ; python recursive_sorting.py

COMPLEXITY COMPARISON TABLE
═══════════════════════════════════════════════════════════════
Algorithm     | Best       | Average    | Worst      | Space      |
──────────────┼────────────┼────────────┼────────────┼────────────┼
Quick Sort    | O(n log n)| O(n log n)| O(n²)      | O(log n)   |
Merge Sort    | O(n log n)| O(n log n)| O(n log n)| O(n)       |
Bubble Sort   | O(n)       | O(n²)      | O(n²)      | O(1)       |
Insertion     | O(n)       | O(n²)      | O(n²)      | O(1)       |
──────────────┴────────────┴────────────┴────────────┴────────────┴


When to Use:
  • Quick Sort:  General purpose, when space is limited, avera
rs
  • Merge Sort:  Need guaranteed O(n log n), stability importa
  • Bubble/Ins:  Educational purposes, nearly sorted small dat


═══════════════════════════════════════════════════════════════
                          TESTING COMPLETE
═══════════════════════════════════════════════════════════════

PS C:\Users\PC\Documents\web projects\CC\HPC>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, n

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

Recommendation Table

Operation	Algorithm	
Search by ID	Hash Map	O(1) lookup
Search by Name	Hash Map	Fast retrieva
Sort by Price	Merge Sort	Stable
Sort by Quantity	Heap Sort	Efficient for

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. Th
gain/loss and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Pri
- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-off

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments
screenshots.

Prompt
Simulate stock price data and sort stocks by percentage change using Heap Sort.
Search stock symbol using hash map. Compare with sorted() and dict.

STUDENT RECORDS MANAGEMENT WITH SORTING ...

Claude Haiku 4.5 • 0.3x

Simulate stock price data and sort stocks by percentage change using Heap Sort.
Search stock symbol using hash map. Compare with sorted() and dict.

codebase

Searched for stock price simulation and created stock_sorter.py

Run `pwsh` command?

```
cd "c:\Users\PC\Documents\web projects\CC\HPC" ; python stock_sorter.py
```

Allow Skip

1 file changed +584 -1 Keep Undo

OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\PC\Documents\web projects\CC\HPC> cd "c:\Users\
ts\CC\HPC" ; python stock_sorter.py


==================================================================
========
                                MARKET STATISTICS

==================================================================
========

Top 10 Gainers:
 1. STK310    | Current: $  210.82 | Change:  +29.91%
 2. STK382    | Current: $  455.84 | Change:  +29.86%
 3. STK546    | Current: $  460.63 | Change:  +29.79%
 4. STK233    | Current: $  319.70 | Change:  +29.71%
 5. STK503    | Current: $  595.22 | Change:  +29.64%
 6. STK498    | Current: $  527.76 | Change:  +29.64%
 7. STK30     | Current: $  373.41 | Change:  +29.58%
 8. STK942    | Current: $  314.72 | Change:  +29.45%
 9. STK71     | Current: $  407.72 | Change:  +29.39%
10. STK783    | Current: $  403.49 | Change:  +29.32%

Top 10 Losers:
 1. STK692    | Current: $  340.09 | Change:  -29.36%
 2. STK139    | Current: $   62.64 | Change:  -29.38%
```

```
Top 10 Losers:
 1. STK692    | Current: $  340.09 | Change:  -29.36%
 2. STK139    | Current: $   62.64 | Change:  -29.38%
 3. STK664    | Current: $  209.83 | Change:  -29.40%
 4. STK29     | Current: $  155.22 | Change:  -29.47%
 5. STK531    | Current: $  330.36 | Change:  -29.66%
 6. STK908    | Current: $  331.15 | Change:  -29.69%
 7. STK373    | Current: $  132.01 | Change:  -29.74%
 8. STK431    | Current: $  202.47 | Change:  -29.82%
 9. STK442    | Current: $  284.60 | Change:  -29.84%
10. STK289    | Current: $   63.49 | Change:  -29.96%

Market Statistics:
  Average Change: +1.17%
  Max Gain:       +29.91%
  Max Loss:       -29.96%
  Range:          59.87%

==================================================================
========

PS C:\Users\PC\Documents\web projects\CC\HPC>
```

AI tools like GitHub Copilot significantly reduce development time, suggest optimized approaches, and help in u
critical evaluation of AI-generated code is required to avoid inefficiencies and logical errors.